

Design Note: LLD Practice Platform

1. Scope Recap

A focused practice loop: **Choose problem** → **Think/design** → **Submit** → **Get feedback** → **Review** → **Try again**. Text-only submissions today, LLM-based rubric evaluation, async processing so the UI never blocks on a slow model call, and non-destructive failure handling (a failed evaluation can be retried without losing the submission). See *Research Note* for the motivating problem.

2. User Flow

- Dashboard** — 3 seeded LLD problems (Parking Lot, Elevator System, Vending Machine).
- Practice Workspace** — left pane: requirements; right pane: free-text design editor (assumptions + design).
- Submit** — the design is validated and persisted synchronously; the API responds `202 Accepted` immediately, and the UI shows an “Evaluating...” state.
- Feedback View** — once evaluation completes: an overall score, a 2-sentence summary, and a per-dimension breakdown (score, evidence, concern, suggestion, confidence).
- History** — an accordion of past attempts per problem, each showing the raw submission, the summary, and the full rubric breakdown, collapsed until clicked. A failed evaluation shows a “Retry Evaluation” action instead of a raw error.

3. Domain Architecture

- Problem** — id, title, description, requirements
- Attempt** — a learner’s attempt at a Problem; a state machine (`submitted` → `evaluating` → `completed` , or `failed` → `evaluating` on retry)
- Submission** — the content a learner submitted, modeled as an interface with a concrete text-blob implementation today (see Change Test A)
- EvaluationEngine** — interface: `evaluate(attemptId, submission, rubric)` ; `LLMEvaluator` is the only implementation wired in today
- Rubric / Criterion** — an ordered list of named dimensions that an EvaluationEngine is asked to score against
- Feedback** — `{ overallScore, summary, results: Result[] }` , where each result is `{ criterion, score, evidence, concern, suggestion, confidence }`

Deterministic validation (is the submission even worth evaluating) is **not** part of this rubric/evaluator machinery at all — it’s a synchronous check at the API boundary (`SubmissionValidator.validateSubmissionText`), rejecting empty or under-50-character text with an HTTP 400 before an Attempt is ever created. This was a deliberate correction during development: an earlier version treated “is the text present” as a rubric criterion sent to the LLM, which mixed trivial checks into real architectural critique in the feedback UI. Splitting it out means:

- the LLM only ever sees and scores submissions worth judging, and
- the two concerns (structural validity vs. architectural quality) can’t bleed into each other in the UI or the data model.

4. Handling Change Test A — New Submission Format

```
interface Submission {
  kind: SubmissionKind; // 'text' today
  isEmpty(): boolean;
  getEvaluableContent(): EvaluableContent; // normalizes for the evaluator
  toPersistedContent(): Record<string, unknown>;
}

interface EvaluableContent {
  text?: string;
  // structuredGraph?: Graph — reserved for a future Class Diagram submission
}
```

The evaluation flow never touches a submission’s raw payload directly — `LLMEvaluator` only ever calls `submission.getEvaluableContent()` and works with the normalized result. Today, `TextSubmission.getEvaluableContent()` returns `{ text }` .

To add a Class Diagram submission (UML, Mermaid, or a parsed AST of the candidate’s diagram) later:

- Implement `ClassDiagramSubmission` satisfying the `Submission` interface, storing whatever structured representation the diagram tool produces.
- Extend `EvaluableContent` with a `structuredGraph` field.
- Teach the evaluator’s prompt-building step to render that structured shape into text an LLM can reason about (e.g., serialize nodes/edges into a class list), or write a dedicated `DiagramEvaluationEngine` if the judgment logic differs enough to warrant it.

No change required to: `Attempt` , its state machine, the API routes, the persistence layer, or the rubric. The `POST /api/attempts` payload gains a `kind` discriminant and a format-specific content field; everything downstream of `submission.getEvaluableContent()` is unaffected.

5. Handling Change Test B — New Evaluator (Strategy Pattern)

```
interface EvaluationEngine {
  readonly kind: 'llm' | 'human' | 'rule_based';
  evaluate(attemptId: string, submission: Submission, rubric: Rubric): Promise<Feedback>;
}
```

`AttemptService` depends only on this interface — it calls `this.llmEvaluator.evaluate(...)` without knowing or caring which concrete class is behind it. `LLMEvaluator` (Gemini-backed, structured JSON output) is the only implementation wired in today, but the contract already anticipates two more:

- **HumanEvaluator** — a reviewer fills in a `Feedback` object through an admin view instead of an LLM call. `evaluate()` would resolve once the reviewer submits their scores (e.g., backed by a polling read on a reviewer-facing table), returning the same `Feedback` shape.
- **RuleBasedEvaluator** — regex/keyword or static-analysis checks (e.g., “does the submission mention an interface for payment,” “is there an if/else chain on a type field”) producing deterministic-but-still-architectural scores, useful as a free tier or as an initial pass before an LLM call.

Plugging either in means writing one new class and constructing `AttemptService` with it (`new AttemptService(problemRepo, new HumanEvaluator())`) — zero changes to `AttemptService`’s orchestration logic, the API routes, or the frontend’s feedback rendering, since every evaluator produces the identical `Feedback` shape the UI already knows how to render.

6. Async State Management

```
submitted → evaluating → completed
                → failed → evaluating (retry)
```

- `POST /api/attempts` runs `SubmissionValidator` synchronously first — a fail-fast, zero-cost rejection that never touches the database or the LLM.
- On success, the `Attempt` row is persisted before evaluation starts (status `submitted`), inside the same request — this guarantees the learner’s work is never lost even if the LLM call times out, errors, or the process restarts.
- The API responds `202 Accepted` immediately with the `attemptId`; a background async function (fire-and-forget, not a real job queue — a deliberate MVP simplification) transitions the attempt to `evaluating`, runs `LLMEvaluator`, and persists the result as `completed` or `failed` with a `failureReason`.
- Failure never deletes data. A failed attempt’s submission stays exactly as it was submitted. `POST /api/attempts/:id/retry` transitions `failed → evaluating` and re-runs the same evaluator against the same persisted submission — the Attempt state machine enforces that only a failed attempt can be retried (`completed` is terminal; retrying a `submitted / evaluating` attempt is rejected with `409`, since it’s already mid-flight).
- The frontend polls `GET /api/attempts/:userId/history` to reflect state changes without blocking on the initial request.

7. Failure & Concurrency Strategy (Practical Scaling Choices)

I had about 5 hours for this build, which isn’t a green light to over-engineer — these are the specific, pragmatic choices I made to keep the system correct under real failure modes without going overboard:

- **Persist before dispatch.** The Postgres insert of the `Attempt` (via Drizzle) happens and is awaited before the evaluation call is ever fired — so a crash, timeout, or an evaluator bug can never lose a submission, only leave it in `evaluating / failed` for the user to retry.
- **Fire-and-forget background execution, not a job queue.** `AttemptService` kicks off evaluation with `void this.processNewAttempt(id).catch(...)` rather than awaiting it in the request handler. This is explicitly a simplification: a real job queue (BullMQ/Redis) was considered and rejected as scope creep for this timebox (see `AI_USAGE.md`) — the states and transitions are written so a real queue could replace the in-process trigger later without changing the state machine itself.
- **Idempotency guards.** Both the fresh-submission path and the retry path check the attempt’s current status before mutating it (`processNewAttempt` only acts on `submitted`; `retryAttempt` only acts on `failed`). A duplicated background invocation is a safe no-op, not a double-charge against the LLM quota.
- **Rate limiting.** `POST /api/attempts` and `POST /api/attempts/:id/retry` share one `express-rate-limit` instance (5 requests/15 min per client by default, configurable via `SUBMIT_RATE_LIMIT`) — both routes trigger a paid/quota-limited Gemini call, so both need the same abuse protection.
- **Resilient evaluator layer.** `LLMEvaluator` wraps the Gemini call with: a 20-second timeout (`AbortController`), up to 2 retries with backoff for a transient 503/overloaded response, `thinkingConfig.thinkingBudget: 0` to stop a reasoning-capable model from burning its output-token budget on internal reasoning and truncating the JSON response mid-object, and a `finishReason` check to catch truncation from any other cause. Every failure mode — quota exceeded, timeout, malformed JSON, service overload — is translated into a specific, human-readable message stored as the attempt’s `failureReason`, never left as an uncaught rejection.